



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

SISTEMAS DISTRIBUIDOS

Laboratorio S09

Mexi Banco: que el aviso no tumbe la transferencia (RabbitMQ)

Nombre del alumno:	Yukioayax Canek Gabriel Hernández
Profesor:	Oscar Manuel Ruiz Hurtado
Grupo:	2
Semestre en curso:	2027-1

Ciudad Universitaria, CDMX

Índice

1. Introducción **2**

2. El problema (Parte 0) **2**

3. Publicar el evento **4**
3.1. Qué se construyó en `transferencia` 4
3.2. Prueba con `notificacion` apagado 5
3.3. ¿Qué garantiza el publisher confirm? 6

4. Desacople en tiempo **6**
4.1. Tres transferencias con `notificacion` apagado 7
4.2. La transferencia revertida 8

5. El duplicado **8**

6. La DLQ **9**
6.1. La configuración 10

7. Preguntas de cierre **11**

8. Conclusiones **11**

1 Introducción

En el laboratorio S08 la transferencia se volvió una saga, pero quedó un cabo suelto: al final llamaba por HTTP a `notificacion` y esperaba su respuesta, en este laboratorio esa llamada síncrona se sustituye por un evento publicado en RabbitMQ: la transferencia deja un recado en el exchange `transferencias` y continúa, y `notificacion` lo lee cuando puede desde su propia cola durable; además se hizo al consumidor idempotente, porque la mensajería entrega al menos una vez, y se configuró una dead letter queue para apartar el mensaje envenenado.

2 El problema (Parte 0)

Se partió de la rama `05-mensajeria-base`. Antes de tocar código, el panel de RabbitMQ solo mostraba los exchanges que trae el broker y ninguna cola.

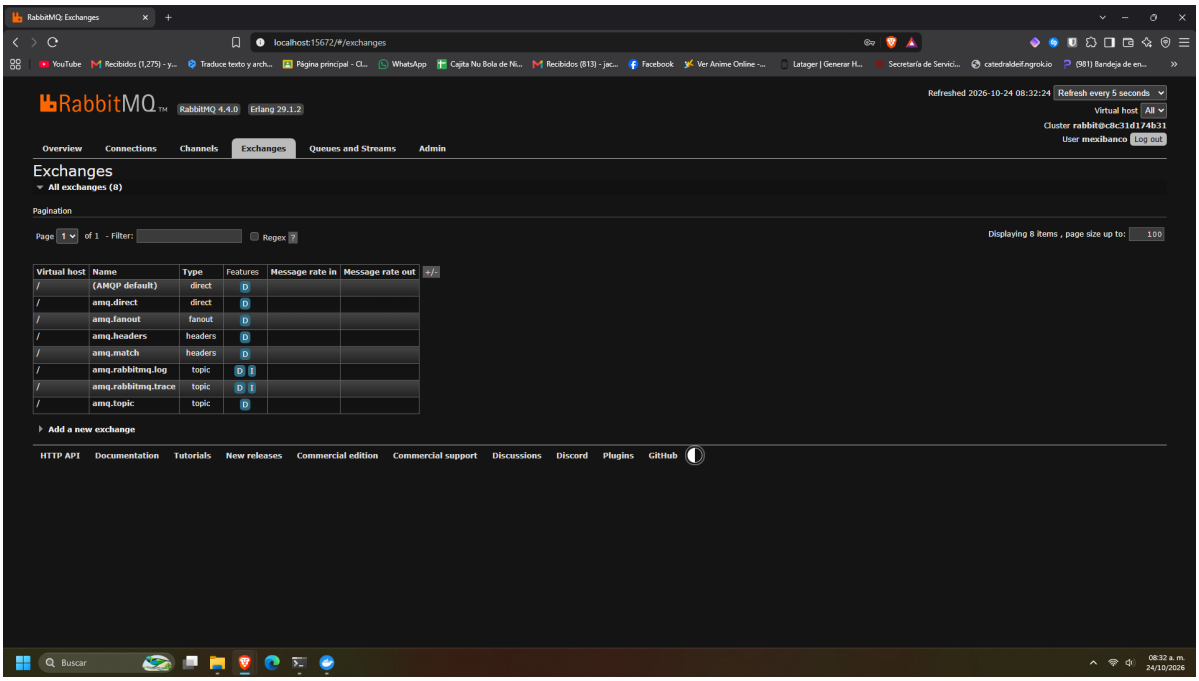


Figura 1: Pestaña Exchanges: solo los exchanges del broker, sin `transferencias`.

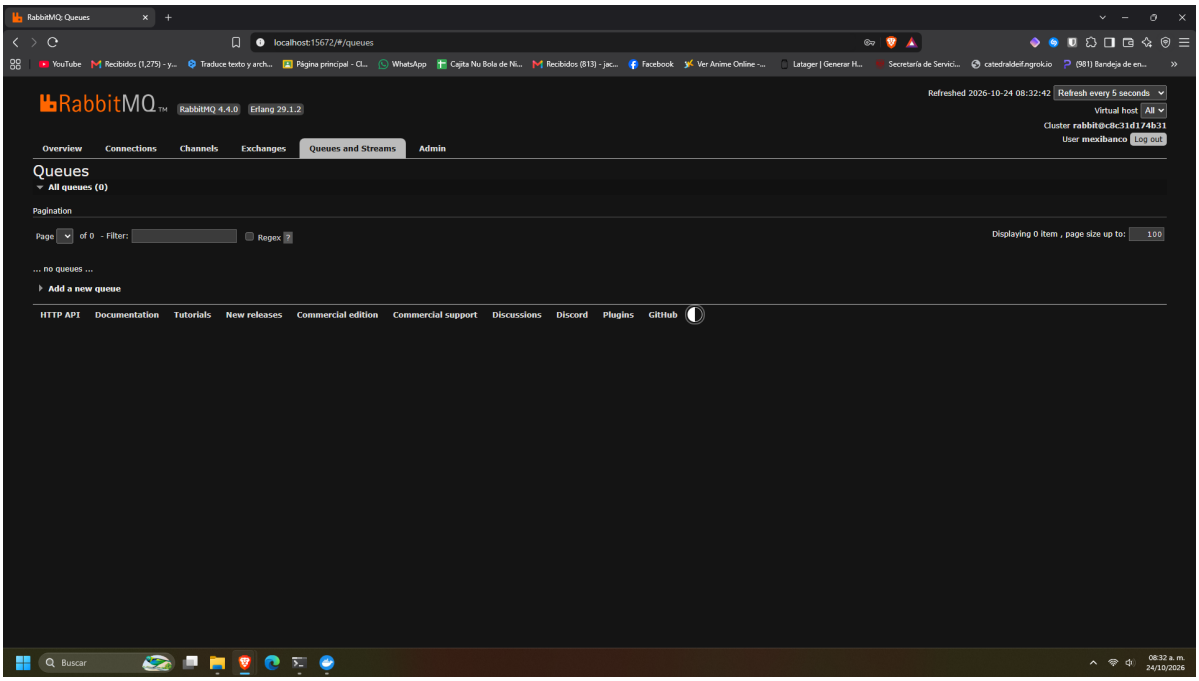


Figura 2: Pestaña Queues and Streams: ninguna cola.

Se crearon las cuentas de Ana (1000) y Beto (500), se apagó `notificacion` y se hizo una transferencia de 50. La respuesta fue un error 503: `Connect timed out` al llamar a `http://notificacion/notificaciones`.



Figura 3: Error 503 de la transferencia con `notificacion` apagado.

Aun así, la transferencia existe y se quedó a medias en estado `ABONADA`:



Figura 4: Consulta por estado `ABONADA`: la transferencia 6 quedó a medias.

Y los saldos sí se movieron: Ana tiene 950 (50 menos) y Beto 550 (50 más).

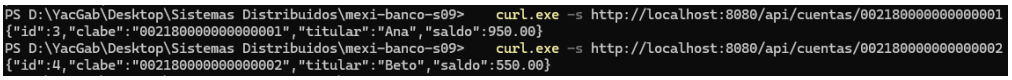


Figura 5: Saldos de Ana y Beto: el dinero ya había cambiado de cuenta.

¿Por qué el cliente recibe un error si el dinero ya se movió? Porque la saga se quedó esperando la respuesta HTTP de `notificacion` después de hacer el cargo, el abono y los asien-

tos; al no contestar, la excepción salió de ese método antes de llegar a `marcarCompletada()`, un aviso, que podía esperar, tumbó una operación que ya estaba hecha.

3 Publicar el evento

3.1 Qué se construyó en `transferencia`

- **`eventoId` en la entidad.** Cada `Transferencia` genera un `UUID` al nacer y lo guarda con su fila (con un `@PostLoad` para las filas viejas que no lo traen), así, si el evento se publica dos veces, lleva el mismo identificador: es la llave de idempotencia del consumidor.
- **`EventoTransferencia`.** Un `record` propio, distinto de la entidad, que es el contrato con quien escuche: un hecho en pasado (`TransferenciaRealizada` o `TransferenciaRevertida`) con su routing key (`transferencia.realizada` o `transferencia.revertida`).
- **Exchange y convertidor JSON.** El exchange `transferencias` es de tipo *topic* y durable; el consumidor decide qué colas enlazar con patrones, los eventos viajan como JSON.
- **Publisher confirms.** Se activaron en `application.yml`:

```
publisher-confirm-type: correlated # el broker contesta ack o nack por mensaje
publisher-returns: true # regresa el mensaje que ninguna cola recibio
template:
  mandatory: true # en lugar de tirarlo en silencio
```

- **`PublicadorDeEventos`.** Publica el evento como mensaje persistente (a disco), con una `CorrelationData` que une la confirmación, que llega después por otro hilo, con ese mensaje, y espera el ack. Si el broker no confirma, lo rechaza (nack) o la routing key no llegó a ninguna cola, lanza `EventoNoPublicadoException`.
- **La saga publica en lugar de llamar.** Se quitaron las dos llamadas HTTP a notificación. Primero se guarda `COMPLETADA` y solo después se publica el evento, para que nadie sea avisado de una transferencia que la base todavía no dice que pasó. En `compensar`, cuando la transferencia queda `COMPENSADA`, se publica `TransferenciaRevertida`.

```
t.marcarCompletada();
t = guardar(t); // primero la base dice COMPLETADA
avisar(EventoTransferencia.realizada(t.getEventoId(), t.getId(), ...)); // luego el
    recado
```

El método `avisar` atrapa cualquier falla al publicar: la transferencia no se revierte, el cliente recibe su 201 y solo se pierde el aviso, que queda en el log.

3.2 Prueba con `notificacion` apagado

Con `notificacion` apagado, la transferencia responde 201 con estado `COMPLETADA` y ya trae su `eventoId`:

```
PS D:\VacGab\Desktop\Sistemas Distribuidos\mexi-banco-s09> curl.exe -i -X POST http://localhost:8080/api/transferencias -H "Content-Type: application/json" -d '{\"claveOrigen\":\"00218000000000001\",\"claveDestino\":\"00218000000000002\",\"monto\":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 15:01:49 GMT

{"id":12,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50,"fecha":"2026-10-24T15:01:49.761933980Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":650.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T15:01:49.864442078Z","eventoId":"466fd4cf-c7ae-4392-abf1-66b576173da2"}
```

Figura 6: Respuesta 201 con estado `COMPLETADA` y `eventoId`, con `notificacion` apagado.

El exchange `transferencias` ya aparece en el panel como *topic* y durable (marca D):

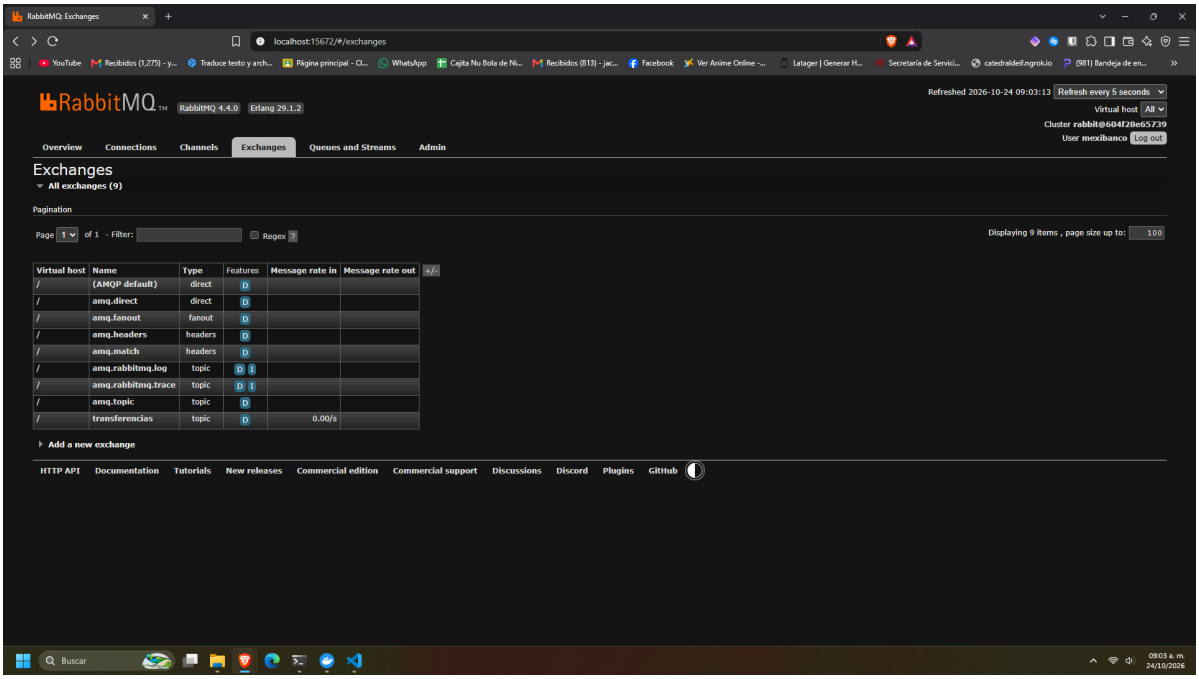


Figura 7: Exchange `transferencias`, de tipo *topic* y durable.

Todavía no existía ninguna cola enlazada, así que el recado no le llegó a nadie. Gracias a `mandatory`, el broker lo regresó y el log de `transferencia` lo dice con `NO_ROUTE`: esos avisos se perdieron.

```
PS D:\VacGab\Desktop\Sistemas Distribuidos\mexi-banco-s09> docker compose logs transferencia | Select-String evento

transferencia-1  | 2026-10-24T15:00:30.877Z ERROR 1 --- [transferencia] [nio-8080-exec-9] m.m.transferencia.TransferenciaService    : Transferencia 8: quedo terminada pero el evento TransferenciaRealizada 0b2c59ea-cad3-42eb-95fe-7e74fe9e10f5 NO se publico: Ninguna cola recibio el evento 0b2c59ea-cad3-42eb-95fe-7e74fe9e10f5 con la routing key transferencia.realizada (NO_ROUTE)
transferencia-1  | 2026-10-24T15:00:47.628Z ERROR 1 --- [transferencia] [nio-8080-exec-4] m.m.transferencia.TransferenciaService    : Transferencia 9: quedo terminada pero el evento TransferenciaRealizada e651ec60-83a2-4fc6-a243-cccc0884b90f NO se publico: Ninguna cola recibio el evento e651ec60-83a2-4fc6-a243-cccc0884b90f con la routing key transferencia.realizada (NO_ROUTE)
transferencia-1  | 2026-10-24T15:00:57.554Z ERROR 1 --- [transferencia] [nio-8080-exec-7] m.m.transferencia.TransferenciaService    : Transferencia 10: quedo terminada pero el evento TransferenciaRealizada 41830453-4ec6-478b-910b-ad0506923f1b NO se publico: Ninguna cola recibio el evento 41830453-4ec6-478b-910b-ad0506923f1b con la routing key transferencia.realizada (NO_ROUTE)
transferencia-1  | 2026-10-24T15:01:45.233Z ERROR 1 --- [transferencia] [nio-8080-exec-7] m.m.transferencia.TransferenciaService    : Transferencia 11: quedo terminada pero el evento TransferenciaRealizada 9d080f84-c43c-4802-915b-40bf58d1ab6e NO se publico: Ninguna cola recibio el evento 9d080f84-c43c-4802-915b-40bf58d1ab6e con la routing key transferencia.realizada (NO_ROUTE)
transferencia-1  | 2026-10-24T15:01:49.875Z ERROR 1 --- [transferencia] [io-8080-exec-10] m.m.transferencia.TransferenciaService   : Transferencia 12: quedo terminada pero el evento TransferenciaRealizada 466fd4cf-c7ae-4392-abf1-66b576173da2 NO se publico: Ninguna cola recibio el evento 466fd4cf-c7ae-4392-abf1-66b576173da2 con la routing key transferencia.realizada (NO_ROUTE)
```

Figura 8: Log de `transferencia`: el evento no se publicó porque ninguna cola lo recibió (`NO_ROUTE`).

Lo mismo pasó al repetir la prueba con tres transferencias: respondieron 201, pero como la cola aún no se había declarado, `rabbitmqctl` no listó ninguna cola y la bandeja de Beto salió vacía al encender `notificacion`, se arregla en la siguiente sección, cuando `notificacion` declara su cola.

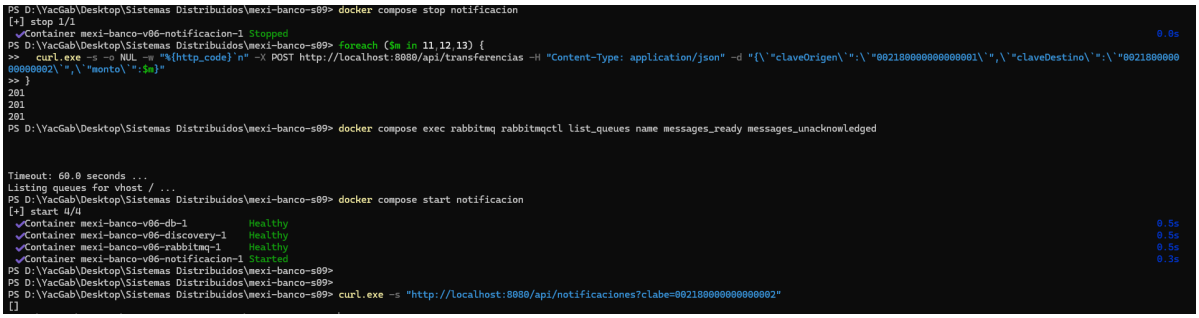


Figura 9: Tres transferencias sin cola declarada: `rabbitmqctl` no lista colas y la bandeja de Beto sale vacía.

3.3 ¿Qué garantiza el publisher confirm?

Qué garantiza: que el broker recibió el mensaje (ack) y, gracias a `mandatory` y `publisher-returns`, que al menos una cola lo recibió; como además es persistente y la cola es durable, sobrevive a un reinicio del broker.
Qué no garantiza:

- Que `notificacion` lo procese. Si está apagado, el mensaje espera en la cola y el confirm solo habla del broker.
- Que el evento salga si el proceso de `transferencia` muere justo entre `guardar(t)` y `avisar(...)`: la base ya dice `COMPLETADA`, pero el evento nunca se publicó y nadie lo reintenta.

4 Desacople en tiempo

Del lado de `notificacion` se construyó: su cola `notificacion.transferencias` (durable, enlazada con `transferencia.*`), su copia local del contrato `EventoTransferencia`, el método `avisarDeTransferencia` (dos avisos por cada `TransferenciaRealizada` y uno por cada `TransferenciaRevertida`, en una sola transacción) y un consumidor con ack manual y `prefetch` de 5.

```
List<Notificacion> avisos = notificaciones.avisarDeTransferencia(evento); // commit
canal.basicAck(etiqueta, false); // DESPUES de guardar
```

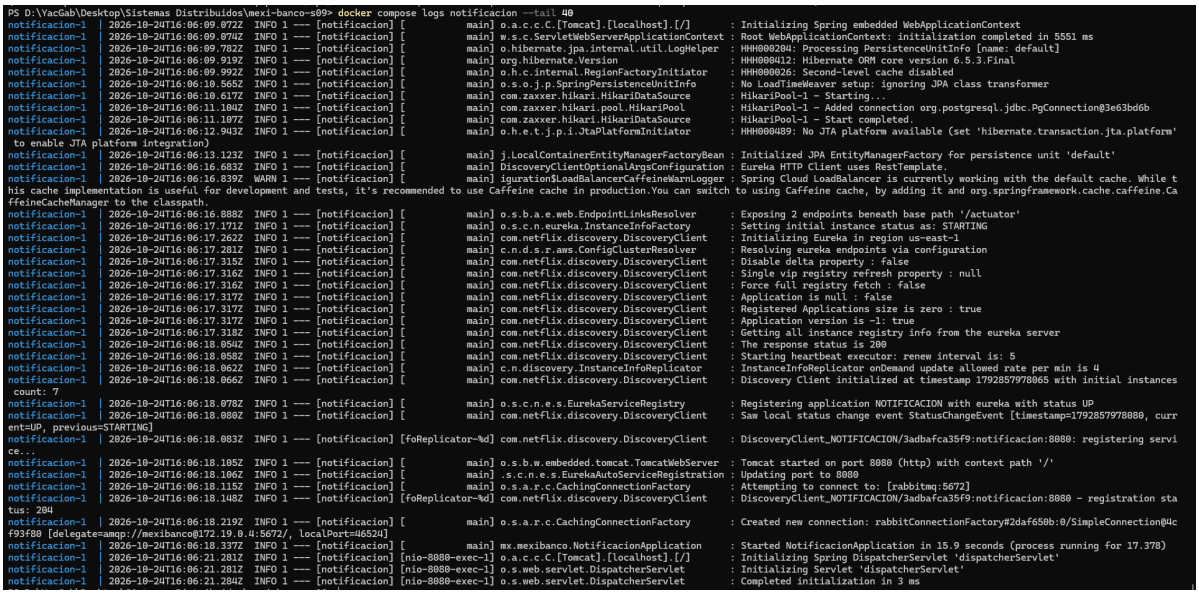


Figura 10: Arranque de `notificacion`: se registra en Eureka y abre su conexión con el broker (`rabbitmq:5672`).

4.1 Tres transferencias con `notificacion` apagado

Con `notificacion` apagado se hicieron tres transferencias (de 11, 12 y 13) y las tres respondieron 201:



Figura 11: Las tres transferencias responden 201 con `notificacion` apagado.

Ahora que la cola existe, guardó los tres recados: 3 *Ready* y 0 *Unacked*.

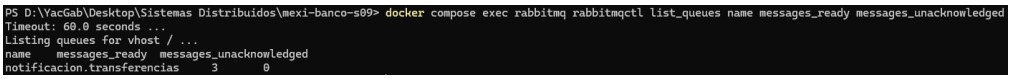


Figura 12: La cola `notificacion.transferencias` con 3 mensajes Ready mientras el consumidor está apagado.

Al encender `notificacion`, el log muestra una línea por evento (transferencias 19, 20 y 21, con `redelivered=false` y 2 avisos nuevos cada una), y la bandeja de Beto tiene `TRANSFERENCIA_RECIBIDA` por 11, 12 y 13. Esto es el desacople en tiempo: productor y consumidor ya no tienen que estar vivos al mismo tiempo.

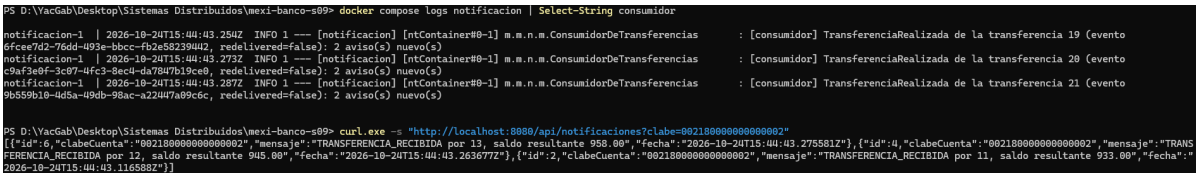


Figura 13: Log del consumidor y bandeja de Beto al volver `notificacion`.

4.2 La transferencia revertida

Se levantó `transferencia` con `FALLAR_ABONO_A_CLABE` apuntando a la cuenta de Beto, para que el abono fallara a propósito, y se transfirieron 150. La respuesta fue un 502 con estado `COMPENSACION_PENDIENTE`: el abono falló como se esperaba, pero en ese momento la devolución del cargo a Ana tampoco se pudo completar, por lo que la saga no llegó a `COMPENSADA`. La API lo dice así: el dinero salió de origen y el banco reintentará la devolución automáticamente. El log de `transferencia` muestra ese reintento programado corriendo cada 10 segundos (ahí para la transferencia 1, una fila vieja de la S08), con el error `Invalid request content`. En la captura el saldo de Ana ya era 542, el mismo que tras la transferencia de 13, y su bandeja todavía no mostraba un aviso `DEVOLUCION_TRANSFERENCIA`, por lo que no se alcanzó a observar el aviso de `TransferenciaRevertida`.

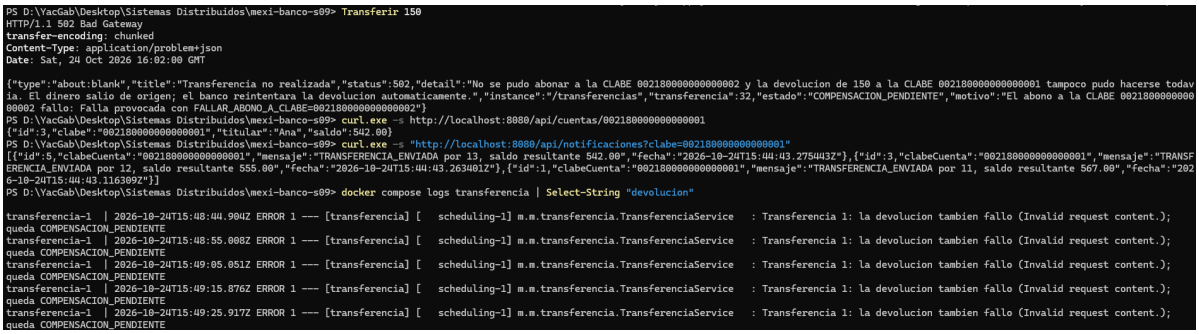


Figura 14: Transferencia de 150 con el abono fallado a propósito: 502 con `COMPENSACION_PENDIENTE`, saldo y bandeja de Ana, y log del reintento de la devolución.

Esto ilustra lo que hace la saga cuando falla también la compensación: la fila no se pierde ni queda en `CARGADA`, sino en `COMPENSACION_PENDIENTE`, y el reintento programado la termina cuando la devolución pase (con la misma clave de idempotencia, para no devolver el dinero dos veces). La causa del `Invalid request content` no se alcanzó a determinar.

5 El duplicado

La mensajería entrega al menos una vez: si `notificacion` muere después de guardar y antes del ack, el broker reentrega el mensaje con `redelivered=true`, y sin protección se guardarían dos juegos de avisos. El arreglo es hacer idempotente a `avisarDeTransferencia`: se creó la tabla `evento_procesado`, con `evento_id` como llave primaria, y al inicio del método se intenta marcar el evento con una sola sentencia:

```
INSERT INTO evento_procesado ... ON CONFLICT (evento_id) DO NOTHING
```

```
// dentro del mismo @Transactional que saveAll(avisos)
if (procesados.marcarSiEsNuevo(evento.eventoId(), evento.tipo(), evento.transferenciaId
    ()) == 0) {
    return List.of(); // ya procesado: 0 avisos nuevos, y el consumidor hace ack
}
```

Es una sola sentencia y no "consultar y luego insertar", porque entre esas dos llamadas otra entrega del mismo mensaje podría colarse. Tampoco se atrapa la violación de llave, porque en Postgres eso deja la transacción abortada. Va en la misma transacción que los avisos para que, si algo falla después de la marca, el rollback se lleve marca y avisos juntos y la siguiente entrega lo intente desde cero. Si el evento ya estaba, el consumidor hace el ack igual y el log dice `0 aviso(s) nuevo(s)`.

> Esta parte quedó implementada en el código entregado (`EventoProcesado`, `EventoProcesadoRepository` y `NotificacionService`), pero no se alcanzó a documentar con capturas el experimento del antes y el después.

6 La DLQ

Un mensaje que nunca se va a poder procesar (*poison pill*) no se arregla por más que se reintente: si se queda en la cola, estorba. La solución es mandarlo a una dead letter queue para que una persona lo revise. Antes del cambio, la cola `notificacion.transferencias` era durable y tenía un consumidor, pero sin argumentos de dead letter:

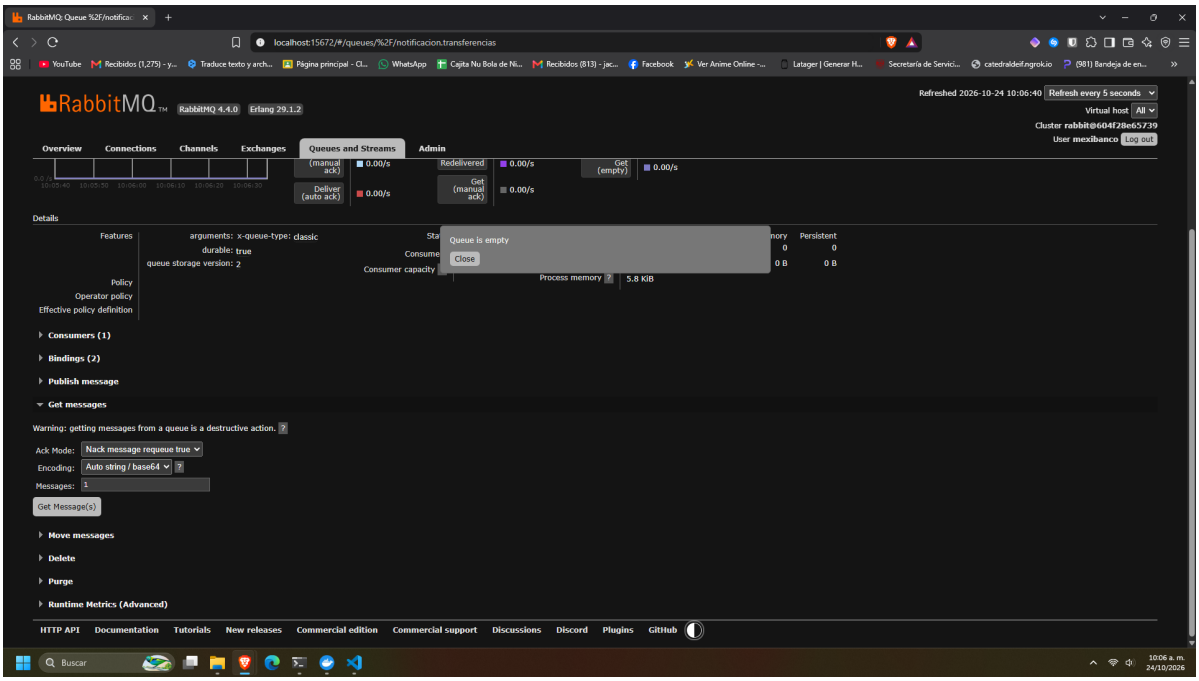


Figura 15: Detalle de la cola antes de la DLQ: durable, un consumidor y sin argumentos `x-dead-letter-*`.

6.1 La configuración

- La cola principal `notificacion.transferencias` lleva los argumentos `x-dead-letter-exchange` = `transferencias.dlx` y `x-dead-letter-routing-key` = `notificacion.transferencias`.
- `transferencias.dlx` es un exchange *direct* y durable.
- `notificacion.transferencias.dlq` es durable y está enlazada al DLX con la clave `notificacion.transferencias`.
- En el consumidor, si el JSON no se puede leer, se hace `basicNack(etiqueta, false, false)`: el último `false` es *requeue*, así que es el broker, no el servicio, quien mueve el mensaje a la DLQ.

Como RabbitMQ no permite cambiar los argumentos de una cola que ya existe, hubo que borrar la cola vieja (estando vacía) y reconstruir `notificacion`; si no, falla con `PRECONDITION_FAILED`.

> La configuración quedó implementada en el código entregado, pero no se alcanzó a documentar con capturas la prueba con el mensaje malo (antes y después).

Para pensar. Un JSON bien formado con un `tipo` que no existe también es veneno: el método lanza `IllegalArgumentException` y, con ack manual, Spring lo reencola y se queda rebotando para siempre, así que también conviene mandarlo a la DLQ. En cambio, si lo que falló fue la base de datos, el mensaje está bien y ahí sí conviene reencolar.

7 Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a `COMPLETADA`, ¿qué pasa con el aviso en mi versión? ¿Por qué no lo arregla el publisher confirm?
La transferencia se completa y el cliente recibe su 201, pero `publicar` falla (por el `connection-timeout` de 2 segundos), `avisar` atrapa la excepción y solo la deja en el log, así que el aviso se pierde y nadie lo reintenta cuando el broker regresa. El publisher confirm no lo arregla porque solo confirma lo que el broker llegó a recibir estando vivo: si no hay broker no hay ack que esperar, y tampoco cubre que el proceso muera entre `guardar(t)` y `avisar(...)`. Es el problema de la doble escritura, porque base y broker son dos sistemas sin una transacción que los una; el Transactional Outbox es la solución.
2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtengo con cada orden?
Si el ack va *antes* de guardar y el consumidor muere en medio, el broker ya borró el mensaje y los avisos no se guardaron: el aviso se pierde (como máximo una vez). Si va *después*, el consumidor muere sin haber confirmado, el broker no recibió el ack y vuelve a entregar el mensaje con `redelivered=true`: no se pierde nada, pero el aviso puede llegar dos veces (al menos una vez). Por eso el ack va después y por eso hace falta la idempotencia por `eventoId`.
3. Elige y di qué sacrificaste.

- **Elección:** reemplazar la llamada HTTP síncrona a `notificacion` por un evento asíncrono en RabbitMQ.
- **Porque:** el aviso puede esperar, y no debe tumbar una transferencia cuyo dinero ya se movió.
- **Sacrifico:** la certeza inmediata de que el aviso salió y llegó una sola vez: ahora hay consistencia eventual, el aviso puede tardar, perderse si el broker está caído o repetirse, y hay que agregar idempotencia, una DLQ y operar un componente más.

8 Conclusiones

La mensajería desacopla a `transferencia` de `notificacion` en el tiempo: la cola durable guarda los recados mientras el consumidor está apagado y el aviso ya no tumba la operación. A cambio, hay que cuidar tres cosas: confirmar el mensaje solo después de guardar, hacer al consumidor idempotente porque la entrega es al menos una vez, y apartar los mensajes que nunca se podrán procesar en una dead letter queue. Queda abierto el caso en que el propio broker está caído, que se resuelve con un Transactional Outbox.